Day08 出行方式识别

> 课程：交通时空大数据分析与可视化系统（Python 版） | 第 8 天（8 课时）
> 主题：从出行段中识别出行方式（地铁/公交/小汽车/其他），结合速度规则和空间匹配

一、本日目标

1. 理解出行方式识别的业务背景和规则设计；
2. 掌握出行段识别（连续出行点合并为出行段）；
3. 掌握速度规则判定（>20km/h地铁或小汽车，>12km/h公交，<12km/h其他）；
4. 掌握出行方式静态数据读取（GBK编码）和预处理；
5. 掌握空间匹配（地铁≥2点、公交≥3点与方式点<500m，否则小汽车）；
6. 完整实现出行方式模块（mode.py）；
7. 分析出行方式结果并可视化；
8. 参数调优与边界情况处理。

二、知识要点

- 出行方式识别基于出行段（连续出行点合并），不是单个点；
- 速度规则：>20km/h → 地铁或小汽车；>12km/h → 公交；<12km/h → 其他；
- 空间匹配：出行段内与地铁站点<500m的点数≥2 → 地铁；与公交站点<500m的点数≥3 → 公交；否则小汽车；
- 静态数据文件为GBK编码，包含地铁和公交站点位置；
- 最终输出：mode_result.csv（出行段信息+出行方式）。

课时1：出行方式识别总览与业务规则

1.1 课时目标

理解出行方式识别的业务背景，掌握四类出行方式的判定规则。

1.2 知识点深度讲解

1.2.1 什么是出行方式

出行方式指用户在出行过程中使用的交通工具。本项目识别四类出行方式：

1. **地铁**：用户乘坐地铁出行，速度快（>20km/h），沿地铁线路；
2. **公交**：用户乘坐公交车出行，速度中等（12-20km/h），沿公交线路；
3. **小汽车**：用户乘坐小汽车（出租车/网约车/私家车），速度快（>20km/h），路线灵活；
4. **其他**：步行、自行车等慢速出行方式（<12km/h）。

1.2.2 为什么需要出行方式识别

出行方式识别是交通时空大数据分析的重要应用：

- **交通规划**：各种出行方式的占比，指导公共交通建设；
- **拥堵分析**：小汽车出行多的区域可能拥堵；
- **环保分析**：公共交通出行比例高，碳排放低；
- **商业分析**：地铁/公交站点附近的商业价值。

1.2.3 判定规则总览

出行方式判定分两步：

1. **速度规则**：根据出行段的平均速度初步判定方式；
2. **空间匹配**：出行段内的点与地铁/公交站点匹配，验证和修正。

速度规则：

速度范围	初步判定	说明
>20 km/h	地铁或小汽车	速度快，可能是地铁或小汽车
12-20 km/h	公交	速度中等，可能是公交
<12 km/h	其他	速度慢，步行/自行车

空间匹配规则：

- 地铁：出行段内与地铁站点<500m的点数 ≥ 2 → 地铁；
- 公交：出行段内与公交站点<500m的点数 ≥ 3 → 公交；
- 小汽车：速度>20km/h 但不满足地铁条件 → 小汽车；
- 其他：速度<12km/h → 其他。

> 为什么地铁需要≥2点、公交需要≥3点？因为地铁站点间距大（约1-2km），出行段内可能只经过2个地铁站；公交站点间距小（约300-500m），出行段内可能经过3个以上公交站。

1.2.4 判定流程

```
出行段
  ↓
计算平均速度
  ↓
速度 < 12km/h → 其他
速度 12-20km/h → 公交（空间匹配验证）
速度 > 20km/h → 地铁或小汽车
  ↓
空间匹配：
  地铁站点匹配点数 ≥ 2 → 地铁
  公交站点匹配点数 ≥ 3 → 公交
  都不满足 → 小汽车
```

1.2.5 与出行目的识别的区别

维度	出行目的	出行方式
分析对象	驻留段	出行段
判定依据	时间规则+空间匹配	速度规则+空间匹配
静态数据	商场/景点POI	地铁/公交站点
匹配阈值	<1km	<500m
输出	居住/购物/旅游/其他	地铁/公交/小汽车/其他

1.3 项目实现步骤

1. 读取阶段3结果（驻留/出行标签）；
2. 识别出行段（连续出行点合并）；
3. 计算出行段平均速度；
4. 应用速度规则初步判定；
5. 读取出行方式静态数据（地铁/公交站点，GBK编码）；
6. 空间匹配（<500m，地铁≥2点、公交≥3点）；
7. 输出结果。

1.4 完整代码（出行方式规则演示）

```
# ============================================================
# 课时1：出行方式识别总览与业务规则
# ============================================================

import pandas as pd
import numpy as np
import os

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
DATA_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\09.交通时空大数据分析系统\09.交通时空大数据分析系统\B-项目数据"

# 读取阶段3结果
df = pd.read_csv(os.path.join(RESULT_DIR, "stage3_label.csv"),
                 dtype={"imsi": str, "time": str, "lac_id": str, "cell_id": str})
print(f"阶段3结果：{len(df)} 行，{df['imsi'].nunique()} 用户")
print(f"  驻留点：{(df['label']==0).sum()}")
print(f"  出行点：{(df['label']==1).sum()}")

# 读取出行方式静态数据（GBK编码）
mode_path = os.path.join(DATA_DIR, "出行方式静态数据.csv")
mode_df = pd.read_csv(mode_path, encoding="gbk")
print(f"\n出行方式静态数据：{len(mode_df)} 行")
print(f"  列：{list(mode_df.columns)}")
print(f"  前5行：")
print(mode_df.head().to_string(index=False))

# 第一列（方式类型）分布
print(f"\n方式类型分布：")
print(mode_df.iloc[:, 0].value_counts())

# 速度规则演示
print(f"\n{'='*60}")
print("速度规则演示")
print(f"{'='*60}")
```

```
def judge_mode_by_speed(avg_speed):
    """根据速度规则初步判定出行方式。"""
    if avg_speed < 12:
        return "其他"
    elif avg_speed <= 20:
        return "公交"
    else:
        return "地铁或小汽车"

# 测试用例
test_cases = [
    (5, "步行"),
    (10, "自行车"),
    (15, "公交"),
    (25, "地铁"),
    (35, "小汽车"),
    (40, "地铁"),
]

print(f"{'速度(km/h)':<12} {'场景':<10} {'初步判定':<15}")
print("-" * 40)
for speed, scene in test_cases:
    mode = judge_mode_by_speed(speed)
    print(f"{speed:<12} {scene:<10} {mode:<15}")

print(f"\n规则说明:")
print(f"  其他: 速度 < 12 km/h（步行/自行车）")
print(f"  公交: 12 km/h <= 速度 <= 20 km/h")
print(f"  地铁或小汽车: 速度 > 20 km/h")
print(f"\n空间匹配规则:")
print(f"  地铁: 出行段内与地铁站点<500m的点数 >= 2")
print(f"  公交: 出行段内与公交站点<500m的点数 >= 3")
print(f"  小汽车: 速度>20km/h 但不满足地铁条件")
print(f"  其他: 速度<12km/h")
```

1.5 运行结果与解读

运行后展示：
- 阶段3结果统计；
- 出行方式静态数据（GBK编码读取，地铁/公交站点）；
- 速度规则测试用例（6个场景的判定结果）。

> 关键理解：
> 1. 出行方式基于出行段，不是单个点；
> 2. 速度规则：<12其他，12-20公交，>20地铁或小汽车；
> 3. 空间匹配：地铁≥2点、公交≥3点（站点间距不同）；
> 4. 静态数据为GBK编码，包含地铁和公交站点位置。

1.6 当堂练习

1. 设计3个边界测试用例（如速度=12、速度=20），验证规则；
2. 思考：为什么地铁需要≥2点而公交需要≥3点？（提示：地铁站点间距大，公交站点间距小）
3. 尝试修改速度阈值（如地铁>25、公交>15），分析对结果的影响。

1.7 课时小结

- 出行方式：地铁、公交、小汽车、其他四类；
- 速度规则：<12其他，12-20公交，>20地铁或小汽车；
- 空间匹配：地铁≥2点、公交≥3点（<500m），否则小汽车；
- 静态数据GBK编码，包含地铁/公交站点；
- 与出行目的识别的区别：分析对象是出行段，依据是速度+空间。

课时2：出行段识别（连续出行点合并）

2.1 课时目标

掌握出行段识别方法：将连续的出行点合并为出行段，计算段的开始/结束时间、持续时间、平均速度。

2.2 知识点深度讲解

2.2.1 什么是出行段

出行段是用户在移动过程中的一段时间，由连续的出行点组成。

例子：
- 用户8:00-8:05在家（驻留点）；
- 用户8:10-8:20在上班路上（5个出行点）→ 一个出行段；
- 用户8:25-12:00在公司（驻留点）。

2.2.2 为什么需要出行段

出行方式是针对"段"的，不是针对"点"的：
- 单个出行点无法计算平均速度；
- 出行段有明确的开始时间、结束时间、持续时间、平均速度，可以应用速度规则；
- 出行段有多个点，可以进行空间匹配（统计匹配站点的点数）。

2.2.3 出行段识别算法

与驻留段识别类似，用 cumsum 实现段ID：

```
df["is_travel"] = df["label"] == 1
df["user_change"] = df["imsi"] != df["imsi"].shift(1)
df["travel_change"] = df["is_travel"] != df["is_travel"].shift(1)
df["seg_change"] = df["user_change"] | df["travel_change"]
df["seg_id"] = df.groupby("imsi")["seg_change"].cumsum()


travel_segments = df[df["is_travel"]].groupby(["imsi", "seg_id"])
```

2.2.4 出行段属性计算

对每个出行段，计算：
1. **开始时间**、**结束时间**、**持续时间**；
2. **点数**；
3. **平均速度**：段内所有相邻点速度的平均值（或总距离/总时间）；
4. **最大速度**、**最小速度**；
5. **总距离**：段内所有相邻点距离之和。

```
segments = travel_df.groupby(["imsi", "seg_id"]).agg(
    start_time=("time", "min"),
    end_time=("time", "max"),
    points=("time", "count"),
    start_lon=("lon", "first"),
    start_lat=("lat", "first"),
    end_lon=("lon", "last"),
    end_lat=("lat", "last"),
).reset_index()
```

> 注意：平均速度需要重新计算（阶段1的速度字段可能没有保存到 stage3_label.csv），需要在出行段内重新计算相邻点的速度。

2.2.5 出行段过滤

有些出行段点数太少（只有1个点），无法计算速度和空间匹配：
- 点数<2的出行段：无法计算平均速度（需要至少2个点）；
- 持续时间太短的出行段：可能是误判。

本项目过滤点数<2的出行段（无法计算速度）。

2.3 项目实现步骤

1. 读取阶段3结果；
2. 用 cumsum 实现出行段ID；
3. 按 imsi-seg_id 分组，计算段属性；
4. 重新计算出行段内的平均速度；
5. 过滤点数<2的出行段；
6. 统计出行段数量和分布；
7. 输出出行段结果。

2.4 完整代码（出行段识别）

```
# ============================================================
# 课时2：出行段识别（连续出行点合并）
# ============================================================

import pandas as pd
import numpy as np
import os

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"


def haversine(lon1, lat1, lon2, lat2, unit="km"):
    """球面距离（向量化）。"""
    R = 6371.0
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))
    return R * c * (1000 if unit == "m" else 1)


# 读取阶段3结果
df = pd.read_csv(os.path.join(RESULT_DIR, "stage3_label.csv"),
                 dtype={"imsi": str, "time": str, "lac_id": str, "cell_id": str})
df = df.sort_values(["imsi", "time"]).reset_index(drop=True)
print(f"数据：{len(df)} 行，{df['imsi'].nunique()} 用户")

# 标记出行点
df["is_travel"] = df["label"] == 1
print(f"出行点：{df['is_travel'].sum()} ({df['is_travel'].mean()*100:.1f}%)")

# 出行段ID
df["user_change"] = df["imsi"] != df["imsi"].shift(1)
df["travel_change"] = df["is_travel"] != df["is_travel"].shift(1)
```

```
df["seg_change"] = df["user_change"] | df["travel_change"]
df["seg_id"] = df.groupby("imsi")["seg_change"].cumsum()

# 只保留出行段
travel_df = df[df["is_travel"]].copy()
print(f"出行段数（含单点段）: {travel_df.groupby(['imsi', 'seg_id']).ngroups}")

# 计算出行段属性
segments = travel_df.groupby(["imsi", "seg_id"]).agg(
    start_time=("time", "min"),
    end_time=("time", "max"),
    points=("time", "count"),
    start_lon=("lon", "first"),
    start_lat=("lat", "first"),
    end_lon=("lon", "last"),
    end_lat=("lat", "last"),
).reset_index()

# 持续时间
segments["start_dt"] = pd.to_datetime(segments["start_time"], format="%Y%m%d%H%M%S",
errors="coerce")
segments["end_dt"] = pd.to_datetime(segments["end_time"], format="%Y%m%d%H%M%S",
errors="coerce")
segments["duration_hours"] = (segments["end_dt"] -
segments["start_dt"]).dt.total_seconds() / 3600

# 重新计算每个出行段内的平均速度
print(f"\n重新计算出行段平均速度...")
avg_speeds = []
max_speeds = []
total_dists = []

for idx, seg in segments.iterrows():
    # 获取该段的所有点
    seg_points = travel_df[(travel_df["imsi"] == seg["imsi"]) & (travel_df["seg_id"] ==
seg["seg_id"])].copy()
    seg_points = seg_points.sort_values("time").reset_index(drop=True)
```

```
    if len(seg_points) < 2:
        avg_speeds.append(np.nan)
        max_speeds.append(np.nan)
        total_dists.append(np.nan)
        continue

    # 计算相邻点的距离和时间差
    seg_points["time_dt"] = pd.to_datetime(seg_points["time"], format="%Y%m%d%H%M%S",
errors="coerce")
    seg_points["time_diff"] = seg_points["time_dt"].diff().dt.total_seconds()
    seg_points["prev_lon"] = seg_points["lon"].shift(1)
    seg_points["prev_lat"] = seg_points["lat"].shift(1)
    seg_points["dist_diff"] = haversine(seg_points["prev_lon"], seg_points["prev_lat"],
                                        seg_points["lon"], seg_points["lat"])
    seg_points["speed"] = seg_points["dist_diff"] * 3600 / seg_points["time_diff"]

    # 过滤异常速度
    valid_speeds = seg_points["speed"][(seg_points["speed"] > 0) & (seg_points["speed"]
< 200)]

    if len(valid_speeds) > 0:
        avg_speeds.append(valid_speeds.mean())
        max_speeds.append(valid_speeds.max())
        total_dists.append(seg_points["dist_diff"].sum())
    else:
        avg_speeds.append(np.nan)
        max_speeds.append(np.nan)
        total_dists.append(np.nan)

segments["avg_speed"] = avg_speeds
segments["max_speed"] = max_speeds
segments["total_dist"] = total_dists

# 过滤点数<2的出行段
before = len(segments)
segments = segments[segments["points"] >= 2].copy()
after = len(segments)
print(f"过滤点数<2的出行段: {before} -> {after} (删除{before-after}段)")
```

```
# 统计
print(f"\n出行段统计:")
print(f"  总段数: {len(segments)}")
print(f"  平均点数: {segments['points'].mean():.1f}")
print(f"  平均持续时间: {segments['duration_hours'].mean():.2f} 小时")
print(f"  平均速度: {segments['avg_speed'].mean():.2f} km/h")
print(f"  速度中位数: {segments['avg_speed'].median():.2f} km/h")

# 速度分布
print(f"\n速度分布:")
bins = [0, 5, 12, 15, 20, 30, 50, 100, 200]
labels_bin = ["<5", "5-12", "12-15", "15-20", "20-30", "30-50", "50-100", "100-200"]
segments["speed_bin"] = pd.cut(segments["avg_speed"], bins=bins, labels=labels_bin,
right=False)
print(segments["speed_bin"].value_counts().sort_index())

# 点数分布
print(f"\n点数分布:")
print(f"  2点段: {(segments['points']==2).sum()}")
print(f"  3-5点段: {((segments['points']>=3)&(segments['points']<=5)).sum()}")
print(f"  6-10点段: {((segments['points']>=6)&(segments['points']<=10)).sum()}")
print(f"  10点以上段: {(segments['points']>10).sum()}")

# 展示前10个出行段
print(f"\n前10个出行段:")
display_cols = ["imsi", "start_time", "end_time", "points", "duration_hours",
                "avg_speed", "max_speed", "total_dist"]
print(segments[display_cols].head(10).to_string(index=False))

# 保存
output_path = os.path.join(RESULT_DIR, "travel_segments.csv")
segments.to_csv(output_path, index=False, encoding="utf-8")
print(f"\n保存: {output_path}")
```

2.5 运行结果与解读

运行后展示：
- 出行段数量和统计（平均点数、平均持续时间、平均速度）；
- 速度分布（8个区间）；
- 点数分布；
- 前10个出行段详情；
- 保存 travel_segments.csv。

> 关键理解：
> 1. 出行段由连续出行点合并而成，驻留点打断；
> 2. 平均速度需要重新计算（阶段1的速度字段可能没有保存）；
> 3. 过滤点数<2的出行段（无法计算速度）；
> 4. 出行段速度分布较广，从<5km/h（步行）到>50km/h（地铁/小汽车）。

2.6 当堂练习

1. 统计平均速度>20km/h的出行段数量，这些可能是地铁或小汽车；
2. 思考：为什么有些出行段平均速度很低（<5km/h）？（提示：可能是步行，或在拥堵路段）
3. 尝试不过滤点数<2的出行段，分析对结果的影响。

2.7 课时小结

- 出行段：连续出行点合并，驻留点打断；
- 段ID用 cumsum 实现；
- 平均速度需要重新计算（相邻点距离/时间）；
- 过滤点数<2的出行段；
- 速度分布较广，从步行到地铁/小汽车。

课时3：速度规则判定（地铁/公交/小汽车/其他）

3.1 课时目标

对每个出行段应用速度规则，初步判定出行方式（其他/公交/地铁或小汽车）。

3.2 知识点深度讲解

3.2.1 速度规则详解

其他（步行/自行车）：
- 平均速度 < 12 km/h；
- 业务含义：用户步行或骑自行车，速度慢。

公交：
- 12 km/h <= 平均速度 <= 20 km/h；
- 业务含义：用户乘坐公交车，速度中等（考虑停站和拥堵）。

地铁或小汽车：
- 平均速度 > 20 km/h；
- 业务含义：用户乘坐地铁或小汽车，速度快；
- 需要空间匹配进一步区分地铁和小汽车。

3.2.2 速度阈值的业务依据

出行方式	典型速度	说明
步行	4-5 km/h	正常步行速度
自行车	10-15 km/h	普通自行车
公交	15-25 km/h	考虑停站和拥堵，平均约20km/h
地铁	30-40 km/h	包括停站，平均约35km/h
小汽车	20-60 km/h	城市内平均约30-40km/h

> 12km/h 是步行/自行车与公交的分界；20km/h 是公交与地铁/小汽车的分界。

3.2.3 判定顺序

```
if avg_speed < 12:
    mode = "其他"
elif avg_speed <= 20:
    mode = "公交"
else:
    mode = "地铁或小汽车"
```

3.2.4 速度规则的局限性

速度规则只能初步判定，可能误判：
- 拥堵时小汽车速度可能<20km/h → 误判为公交；
- 快速公交（BRT）速度可能>20km/h → 误判为地铁或小汽车；
- 步行+地铁组合出行段平均速度可能在12-20之间 → 误判为公交。

空间匹配可以修正部分误判。

3.3 项目实现步骤

1. 读取出行段结果；
2. 对每个出行段应用速度规则；
3. 统计初步判定结果；
4. 按速度区间分析；
5. 输出带速度规则判定的出行段结果。

3.4 完整代码（速度规则判定）

```
# ============================================================
# 课时3：速度规则判定（地铁/公交/小汽车/其他）
# ============================================================

import pandas as pd
import numpy as np
import os

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"

# 读取出行段
segments = pd.read_csv(os.path.join(RESULT_DIR, "travel_segments.csv"),
                       dtype={"imsi": str, "start_time": str, "end_time": str})
print(f"出行段：{len(segments)} 段")

# 过滤速度为空的段
before = len(segments)
segments = segments[segments["avg_speed"].notna()].copy()
after = len(segments)
print(f"过滤速度为空：{before} -> {after}")


def judge_mode_by_speed(avg_speed):
    """根据速度规则初步判定出行方式。"""
    if avg_speed < 12:
        return "其他"
    elif avg_speed <= 20:
        return "公交"
    else:
        return "地铁或小汽车"


# 应用速度规则
segments["mode_by_speed"] = segments["avg_speed"].apply(judge_mode_by_speed)
```

```
# 统计
print(f"\n速度规则初步判定结果:")
print(segments["mode_by_speed"].value_counts())
print(f"\n比例:")
print((segments["mode_by_speed"].value_counts(normalize=True) * 100).round(1))

# 按方式统计速度
print(f"\n按方式统计速度:")
print(f"{'方式':<15} {'段数':<8} {'平均速度':<12} {'中位速度':<12} {'速度范围':<20}")
print("-" * 60)
for mode in ["其他", "公交", "地铁或小汽车"]:
    sub = segments[segments["mode_by_speed"] == mode]
    if len(sub) > 0:
        speed_range = f"{sub['avg_speed'].min():.1f}-{sub['avg_speed'].max():.1f}"
        print(f"{mode:<15} {len(sub):<8} {sub['avg_speed'].mean():<12.2f} "
              f"{sub['avg_speed'].median():<12.2f} {speed_range:<20}")

# 边界情况分析
print(f"\n边界情况分析:")
# 速度接近12
near_12 = segments[(segments["avg_speed"] >= 10) & (segments["avg_speed"] <= 14)]
print(f"  速度接近12(10-14): {len(near_12)} 段")
print(f"    其他: {(near_12['mode_by_speed']=='其他').sum()}, 公交:
{(near_12['mode_by_speed']=='公交').sum()}")

# 速度接近20
near_20 = segments[(segments["avg_speed"] >= 18) & (segments["avg_speed"] <= 22)]
print(f"  速度接近20(18-22): {len(near_20)} 段")
print(f"    公交: {(near_20['mode_by_speed']=='公交').sum()}, 地铁或小汽车:
{(near_20['mode_by_speed']=='地铁或小汽车').sum()}")

# 高速段（>50）
high_speed = segments[segments["avg_speed"] > 50]
print(f"  高速段(>50km/h): {len(high_speed)} 段")
print(f"    最大速度: {high_speed['avg_speed'].max():.1f} km/h")

# 展示各类的示例
print(f"\n各类的示例（各3个）:")
```

```
for mode in ["其他", "公交", "地铁或小汽车"]:
    sub = segments[segments["mode_by_speed"] == mode].head(3)
    print(f"\n  【{mode}】")
    print(sub[["imsi", "start_time", "points", "duration_hours",
"avg_speed"]].to_string(index=False))

# 保存
output_path = os.path.join(RESULT_DIR, "travel_segments_with_mode_speed.csv")
segments.to_csv(output_path, index=False, encoding="utf-8")
print(f"\n保存: {output_path}")
```

3.5 运行结果与解读

运行后展示:
- 速度规则初步判定结果（三类的数量和比例）；
- 按方式统计速度（平均、中位、范围）；
- 边界情况分析（速度接近12、接近20、高速段）；
- 各类的示例;
- 保存 travel_segments_with_mode_speed.csv。

> 关键理解:
> 1. 速度规则判定后，"其他"通常最多（很多出行段速度慢，可能是步行）；
> 2. 公交段速度在12-20km/h之间;
> 3. 地铁或小汽车段速度>20km/h，需要空间匹配进一步区分;
> 4. 边界速度（接近12或20）的段可能误判，需要空间匹配修正。

3.6 当堂练习

1. 统计"地铁或小汽车"段中速度>30km/h的数量，这些更可能是地铁;
2. 思考：为什么"其他"类通常最多？（提示：城市内步行出行段多，或出行段速度计算不准确）
3. 尝试把公交速度上限改为25km/h，对比公交段数量变化。

3.7 课时小结

- 速度规则：<12其他，12-20公交，>20地铁或小汽车;
- 12km/h是步行/自行车与公交的分界，20km/h是公交与地铁/小汽车的分界;
- "其他"通常最多，公交次之，地铁或小汽车较少;
- 边界速度可能误判，需要空间匹配修正;

- 输出 travel_segments_with_mode_speed.csv。

课时4：出行方式静态数据读取与预处理

4.1 课时目标

掌握出行方式静态数据（地铁/公交站点）的读取方法（GBK编码），理解数据结构和预处理。

4.2 知识点深度讲解

4.2.1 静态数据文件

出行方式静态数据文件：`出行方式静态数据.csv`
- 编码：GBK；
- 列数：5列；
- 行数：约968行；
- 内容：地铁站点和公交站点的位置信息。

4.2.2 数据结构

出行方式静态数据有5列（具体列名需读取后确认）：
1. **方式类型**：地铁/公交；
2. **站点名称**：站点名称；
3. **线路名称**：所属线路（如地铁1号线、公交100路）；
4. **经度**：站点经度；
5. **纬度**：站点纬度。

> 注意：不同项目的静态数据列名可能不同，读取后需用 `df.columns` 确认。

4.2.3 地铁 vs 公交站点的区别

特征	地铁站点	公交站点
数量	较少（约几十到一百）	较多（约几百到一千）
间距	大（约1-2km）	小（约300-500m）
速度	快（30-40km/h）	中（15-25km/h）
匹配点数阈值	≥2	≥3

> 因为地铁站点间距大，出行段内可能只经过2个地铁站；公交站点间距小，出行段内可能经过3个以上公交站。

4.2.4 预处理

1. **列名重命名**：统一为 `mode_type, station_name, line_name, lon, lat`；
2. **经纬度类型转换**：确保为数值型；
3. **去重**：去除重复的站点（同一站点可能属于多条线路）；
4. **过滤无效值**：去除经纬度为空或异常的行；
5. **分离地铁和公交**：按方式类型分为两个DataFrame，方便空间匹配。

4.3 项目实现步骤

1. 用GBK编码读取出行方式静态数据；
2. 查看数据结构（列名、行数、前几行）；
3. 预处理（列名重命名、类型转换、去重）；
4. 分离地铁和公交站点；
5. 统计地铁和公交站点数量；
6. 保存预处理后的静态数据。

4.4 完整代码（静态数据读取与预处理）

```
# ============================================================
# 课时4：出行方式静态数据读取与预处理
# ============================================================

import pandas as pd
import numpy as np
import os

DATA_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\09.交通时空大数据分析系统\09.交通时空大数据分析系统\B-项目数据"
RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"

# 读取出行方式静态数据（GBK编码）
mode_path = os.path.join(DATA_DIR, "出行方式静态数据.csv")

print("=" * 60)
print("读取出行方式静态数据")
print("=" * 60)

try:
    mode_df = pd.read_csv(mode_path, encoding="gbk")
    print(f"成功用 GBK 读取")
except UnicodeDecodeError:
    print("GBK 读取失败，尝试其他编码...")
    for enc in ["utf-8", "gb2312", "latin1"]:
        try:
            mode_df = pd.read_csv(mode_path, encoding=enc)
            print(f"成功用 {enc} 读取")
            break
        except UnicodeDecodeError:
            continue

print(f"  行数：{len(mode_df)}")
print(f"  列数：{len(mode_df.columns)}")
print(f"  列名：{list(mode_df.columns)}")
```

```
print(f"\n前10行:")
print(mode_df.head(10).to_string(index=False))

# 数据类型
print(f"\n数据类型:")
print(mode_df.dtypes)

# 缺失值
print(f"\n缺失值统计:")
print(mode_df.isnull().sum())

# 第一列（方式类型）分布
print(f"\n方式类型分布:")
print(mode_df.iloc[:, 0].value_counts())

# 预处理
print(f"\n{'='*60}")
print("数据预处理")
print(f"{'='*60}")

# 重命名列（假设5列：类型、名称、线路、经度、纬度）
if len(mode_df.columns) == 5:
    mode_df.columns = ["mode_type", "station_name", "line_name", "lon", "lat"]
    print(f"列重命名为: {list(mode_df.columns)}")
else:
    print(f"列数不是5，保持原列名: {list(mode_df.columns)}")

# 经纬度转换为数值型
mode_df["lon"] = pd.to_numeric(mode_df["lon"], errors="coerce")
mode_df["lat"] = pd.to_numeric(mode_df["lat"], errors="coerce")

# 去除经纬度缺失的行
before = len(mode_df)
mode_df = mode_df.dropna(subset=["lon", "lat"])
after = len(mode_df)
print(f"去除经纬度缺失: {before} -> {after} (删除{before-after}行)")

# 去重
```

```
before = len(mode_df)
mode_df = mode_df.drop_duplicates()
after = len(mode_df)
print(f"去重: {before} -> {after} (删除{before-after}行)")

# 经纬度范围检查
print(f"\n经纬度范围:")
print(f"  经度: {mode_df['lon'].min():.4f} ~ {mode_df['lon'].max():.4f}")
print(f"  纬度: {mode_df['lat'].min():.4f} ~ {mode_df['lat'].max():.4f}")

# 分离地铁和公交
print(f"\n分离地铁和公交:")
# 方式类型可能是"地铁"/"公交"或其他名称，需要确认
mode_types = mode_df["mode_type"].unique()
print(f"  方式类型: {mode_types.tolist()}")

# 简单匹配（包含"地铁"的为地铁，包含"公交"的为公交）
subway_df = mode_df[mode_df["mode_type"].str.contains("地铁", na=False)].copy()
bus_df = mode_df[mode_df["mode_type"].str.contains("公交", na=False)].copy()

print(f"  地铁站点: {len(subway_df)}")
print(f"  公交站点: {len(bus_df)}")

# 地铁线路统计
if len(subway_df) > 0:
    print(f"\n地铁线路统计:")
    print(subway_df["line_name"].value_counts().head(10))

# 公交线路统计
if len(bus_df) > 0:
    print(f"\n公交线路统计（Top10）:")
    print(bus_df["line_name"].value_counts().head(10))

# 保存预处理后的静态数据
output_path = os.path.join(RESULT_DIR, "mode_stations_cleaned.csv")
mode_df.to_csv(output_path, index=False, encoding="utf-8")
print(f"\n保存全部站点: {output_path} ({len(mode_df)} 行)")
```

```
subway_path = os.path.join(RESULT_DIR, "subway_stations.csv")
subway_df.to_csv(subway_path, index=False, encoding="utf-8")
print(f"保存地铁站点：{subway_path} ({len(subway_df)} 行)")

bus_path = os.path.join(RESULT_DIR, "bus_stations.csv")
bus_df.to_csv(bus_path, index=False, encoding="utf-8")
print(f"保存公交站点：{bus_path} ({len(bus_df)} 行)")

print(f"\n结论：")
print(f"  1. 出行方式静态数据为GBK编码，读取时必须指定 encoding='gbk'")
print(f"  2. 数据包含方式类型、站点名称、线路名称、经度、纬度5列")
print(f"  3. 地铁站点较少（间距大），公交站点较多（间距小）")
print(f"  4. 预处理后分离地铁和公交，方便空间匹配")
```

4.5 运行结果与解读

运行后展示：
- 静态数据读取结果（行数、列名、前10行）；
- 数据类型和缺失值统计；
- 方式类型分布；
- 预处理（列重命名、类型转换、去重、缺失值处理）；
- 经纬度范围检查；
- 地铁和公交站点分离及线路统计；
- 保存预处理后的静态数据。

> 关键理解：
> 1. 出行方式静态数据为GBK编码，读取时必须指定 `encoding="gbk"` ；
> 2. 数据包含5列（方式类型、站点名称、线路名称、经度、纬度）；
> 3. 地铁站点较少（间距大，约1-2km），公交站点较多（间距小，约300-500m）；
> 4. 预处理后分离地铁和公交，方便空间匹配。

4.6 当堂练习

1. 统计地铁站点和公交站点的经纬度范围，分析是否都在沈阳市内；
2. 思考：为什么地铁站点间距比公交大？（提示：地铁建设成本高，站点间距大；公交灵活，站点间距小）
3. 尝试找出重复的站点（同一站点属于多条线路），分析去重策略。

4.7 课时小结

- 静态数据GBK编码，5列（类型、名称、线路、经度、纬度），约968行；
- 地铁站点少（间距大），公交站点多（间距小）；
- 预处理：列重命名、类型转换、去重、缺失值处理；
- 分离地铁和公交，方便空间匹配；
- 保存 mode_stations_cleaned.csv、subway_stations.csv、bus_stations.csv。

课时5：空间匹配（地铁≥2点、公交≥3点，<500m）

5.1 课时目标

掌握出行方式空间匹配方法：对每个出行段，统计段内与地铁/公交站点<500m的点数，根据阈值判定地铁或公交。

5.2 知识点深度讲解

5.2.1 空间匹配的目的

速度规则只能区分"其他/公交/地铁或小汽车"，无法区分地铁和小汽车。空间匹配通过统计出行段内经过的地铁/公交站点数量，进一步区分：
- 出行段内经过≥2个地铁站 → 地铁；
- 出行段内经过≥3个公交站 → 公交；
- 都不满足且速度>20km/h → 小汽车。

5.2.2 空间匹配算法

对每个出行段：
1. 获取段内所有点的经纬度；
2. 对每个点，计算到所有地铁站点的距离，判断是否<500m；
3. 统计段内与地铁站<500m的点数（去重：同一个点可能匹配多个地铁站）；
4. 同理统计与公交站<500m的点数；
5. 应用阈值：地铁≥2点 → 地铁；公交≥3点 → 公交。

```
# 对出行段的每个点
for point in segment_points:
    # 到所有地铁站的距离
    dists_to_subway = haversine(point_lon, point_lat, subway_lons, subway_lats,
unit="m")
    # 是否有地铁站<500m
    if (dists_to_subway < 500).any():
        subway_match_count += 1
```

5.2.3 <500m 阈值的业务含义

为什么是500m?

- 基站定位误差：城市内基站定位误差约200-500m；
- 站点覆盖范围：地铁站/公交站的步行覆盖范围约500m；
- 500m 是合理的匹配半径（允许定位误差，同时避免匹配到太远的站点）。

5.2.4 地铁≥2点、公交≥3点的业务依据

地铁≥2点：

- 地铁站点间距大（约1-2km）；
- 一个出行段（持续几分钟到几十分钟）可能只经过2个地铁站；
- ≥2点可以确认用户确实在地铁线路上（不是偶然经过一个地铁站附近）。

公交≥3点：

- 公交站点间距小（约300-500m）；
- 一个出行段可能经过3个以上公交站；
- ≥3点可以确认用户确实在公交线路上。

> 如果地铁阈值设为≥3点，很多地铁出行段可能只经过2个地铁站，会被误判为小汽车；如果公交阈值设为≥2点，很多步行出行段可能偶然经过2个公交站，会被误判为公交。

5.2.5 判定优先级

当一个出行段同时满足地铁和公交条件时（既经过≥2个地铁站，又经过≥3个公交站），如何判定?

优先级：地铁 > 公交 > 小汽车

- 因为地铁速度更快（>20km/h），如果同时满足地铁条件，优先判为地铁；
- 公交速度中等（12-20km/h），如果不满足地铁但满足公交，判为公交；
- 小汽车：速度>20km/h 但不满足地铁条件。

```
if subway_match_count >= 2:
    final_mode = "地铁"
elif bus_match_count >= 3:
    final_mode = "公交"
elif avg_speed > 20:
    final_mode = "小汽车"
else:
    final_mode = "其他"
```

5.2.6 性能优化

如果出行段数量多（如几千段），每段有多个点，每个点要计算到所有站点的距离，时间复杂度较高。

优化方法:
1. **向量化**：一次计算一个点到所有站点的距离（已实现）；
2. **粗筛**：先用经纬度范围粗筛（如经度差<0.005度≈500m），再精确计算；
3. **空间索引**：用 KDTree 加速最近邻搜索。

本项目数据量不大（出行段约100-200段），向量化计算足够快。

5.3 项目实现步骤

1. 读取出行段结果（含速度规则判定）和地铁/公交站点数据；
2. 对每个出行段，获取段内所有点；
3. 统计段内与地铁站<500m的点数；
4. 统计段内与公交站<500m的点数；
5. 应用阈值和优先级，判定最终出行方式；
6. 统计最终出行方式分布；
7. 输出结果。

5.4 完整代码（空间匹配）

```
# ============================================================
# 课时5：空间匹配（地铁≥2点、公交≥3点，<500m）
# ============================================================

import pandas as pd
import numpy as np
import os

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"


def haversine(lon1, lat1, lon2, lat2, unit="km"):
    """球面距离（向量化）。"""
    R = 6371.0
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))
    return R * c * (1000 if unit == "m" else 1)


# 读取阶段3结果（用于获取出行段内的点）
df = pd.read_csv(os.path.join(RESULT_DIR, "stage3_label.csv"),
                 dtype={"imsi": str, "time": str, "lac_id": str, "cell_id": str})
df = df.sort_values(["imsi", "time"]).reset_index(drop=True)

# 出行段ID
df["is_travel"] = df["label"] == 1
df["user_change"] = df["imsi"] != df["imsi"].shift(1)
df["travel_change"] = df["is_travel"] != df["is_travel"].shift(1)
df["seg_change"] = df["user_change"] | df["travel_change"]
df["seg_id"] = df.groupby("imsi")["seg_change"].cumsum()
travel_df = df[df["is_travel"]].copy()
```

```
# 读取出行段（含速度规则判定）
segments = pd.read_csv(os.path.join(RESULT_DIR, "travel_segments_with_mode_speed.csv"),
                       dtype={"imsi": str, "start_time": str, "end_time": str})
print(f"出行段: {len(segments)} 段")

# 读取地铁和公交站点
subway_df = pd.read_csv(os.path.join(RESULT_DIR, "subway_stations.csv"))
bus_df = pd.read_csv(os.path.join(RESULT_DIR, "bus_stations.csv"))
print(f"地铁站点: {len(subway_df)}, 公交站点: {len(bus_df)}")

# 站点经纬度数组
subway_lons = subway_df["lon"].values
subway_lats = subway_df["lat"].values
bus_lons = bus_df["lon"].values
bus_lats = bus_df["lat"].values

# 空间匹配
MATCH_THRESHOLD = 500  # 匹配阈值（m）
SUBWAY_MIN_POINTS = 2  # 地铁最小匹配点数
BUS_MIN_POINTS = 3     # 公交最小匹配点数

print(f"\n开始空间匹配（阈值={MATCH_THRESHOLD}m, 地铁≥{SUBWAY_MIN_POINTS}点, 公交
≥{BUS_MIN_POINTS}点）...")

# 初始化匹配结果列
segments["subway_match_points"] = 0
segments["bus_match_points"] = 0
segments["final_mode"] = None

for idx, seg in segments.iterrows():
    # 获取该出行段的所有点
    seg_points = travel_df[(travel_df["imsi"] == seg["imsi"]) &
                           (travel_df["seg_id"] == seg["seg_id"])].copy()
    seg_points = seg_points.sort_values("time").reset_index(drop=True)

    if len(seg_points) == 0:
        continue
```

```
    # 统计与地铁站<500m的点数
    subway_count = 0
    for _, point in seg_points.iterrows():
        dists = haversine(point["lon"], point["lat"], subway_lons, subway_lats,
unit="m")
        if (dists < MATCH_THRESHOLD).any():
            subway_count += 1

    # 统计与公交站<500m的点数
    bus_count = 0
    for _, point in seg_points.iterrows():
        dists = haversine(point["lon"], point["lat"], bus_lons, bus_lats, unit="m")
        if (dists < MATCH_THRESHOLD).any():
            bus_count += 1

    segments.at[idx, "subway_match_points"] = subway_count
    segments.at[idx, "bus_match_points"] = bus_count

    # 最终判定（优先级：地铁 > 公交 > 小汽车 > 其他）
    if subway_count >= SUBWAY_MIN_POINTS:
        final_mode = "地铁"
    elif bus_count >= BUS_MIN_POINTS:
        final_mode = "公交"
    elif seg["avg_speed"] > 20:
        final_mode = "小汽车"
    else:
        final_mode = "其他"

    segments.at[idx, "final_mode"] = final_mode

    # 进度
    if (idx + 1) % 50 == 0:
        print(f"  已处理 {idx+1}/{len(segments)} 段...")

# 最终出行方式分布
print(f"\n最终出行方式分布:")
print(segments["final_mode"].value_counts())
print(f"\n比例:")
```

```
print((segments["final_mode"].value_counts(normalize=True) * 100).round(1))

# 匹配点数统计
print(f"\n地铁匹配点数统计:")
print(f"  0点: {(segments['subway_match_points']==0).sum()}")
print(f"  1点: {(segments['subway_match_points']==1).sum()}")
print(f"  ≥2点: {(segments['subway_match_points']>=2).sum()}")

print(f"\n公交匹配点数统计:")
print(f"  0点: {(segments['bus_match_points']==0).sum()}")
print(f"  1-2点: {((segments['bus_match_points']>=1)&(segments['bus_match_points']
<=2)).sum()}")
print(f"  ≥3点: {(segments['bus_match_points']>=3).sum()}")

# 速度规则 vs 最终方式对比
print(f"\n速度规则 vs 最终方式对比:")
cross = pd.crosstab(segments["mode_by_speed"], segments["final_mode"], margins=True)
print(cross)

# 展示各类的示例
print(f"\n各类的示例（各3个）:")
for mode in ["地铁", "公交", "小汽车", "其他"]:
    sub = segments[segments["final_mode"] == mode].head(3)
    print(f"\n  【{mode}】")
    print(sub[["imsi", "start_time", "points", "avg_speed",
               "subway_match_points", "bus_match_points"]].to_string(index=False))

# 保存
output_path = os.path.join(RESULT_DIR, "travel_segments_with_match.csv")
segments.to_csv(output_path, index=False, encoding="utf-8")
print(f"\n保存: {output_path}")
```

5.5 运行结果与解读

运行后展示:
- 最终出行方式分布（四类的数量和比例）；
- 地铁/公交匹配点数统计;
- 速度规则 vs 最终方式对比（交叉表）；

- 各类的示例；
- 保存 travel_segments_with_match.csv。

> 关键理解：
> 1. 空间匹配后，"地铁或小汽车"被进一步区分为地铁和小汽车；
> 2. 地铁段通常匹配≥2个地铁站，公交段匹配≥3个公交站；
> 3. 小汽车段速度>20km/h但不满足地铁/公交条件；
> 4. 其他段速度<12km/h；
> 5. 交叉表展示速度规则和最终方式的差异（空间匹配修正了部分误判）。

5.6 当堂练习

1. 统计地铁段的平均速度，与小汽车段对比，分析差异；
2. 思考：为什么有些公交段速度>20km/h？（提示：快速公交BRT，或公交在高速路段）
3. 尝试把地铁阈值改为≥3点，对比地铁段数量变化，分析哪个更合理。

5.7 课时小结

- 空间匹配：出行段内与站点<500m的点数统计；
- 地铁≥2点、公交≥3点（站点间距不同）；
- 判定优先级：地铁 > 公交 > 小汽车 > 其他；
- 500m阈值允许定位误差；
- 空间匹配修正速度规则的误判，区分地铁和小汽车。

课时6：出行方式完整实现（mode.py）

6.1 课时目标

将出行方式识别所有逻辑整合为完整模块 mode.py。

6.2 知识点深度讲解

6.2.1 模块结构

mode.py 的结构：
1. 配置区（路径、阈值）；
2. 工具函数（haversine、速度规则判定）；

3. 主函数 mode_analysis();
4. 入口。

6.2.2 完整流程

```
读取 stage3_label.csv
  ↓
出行段识别（连续出行点合并）
  ↓
计算出行段平均速度
  ↓
速度规则判定（其他/公交/地铁或小汽车）
  ↓
读取出行方式静态数据（GBK）
  ↓
空间匹配（<500m，地铁≥2点、公交≥3点）
  ↓
最终出行方式判定
  ↓
输出 mode_result.csv
```

6.3 项目实现步骤

按上述流程编写完整代码。

6.4 完整代码（mode.py）

```
# ============================================================
# 出行方式模块：mode.py
# 功能：出行段识别、速度规则、空间匹配、最终判定
# 运行：python mode.py
# 输入：stage3_label.csv，出行方式静态数据.csv
# 输出：mode_result.csv
# ============================================================

import os
import pandas as pd
import numpy as np

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
DATA_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\09.交通时空大数据分析系统\09.交通时空大数据分析系统\B-项目数据"
os.makedirs(RESULT_DIR, exist_ok=True)

# 阈值
MATCH_THRESHOLD = 500   # 空间匹配阈值（m）
SUBWAY_MIN_POINTS = 2   # 地铁最小匹配点数
BUS_MIN_POINTS = 3      # 公交最小匹配点数
SPEED_OTHER = 12        # 其他速度上限（km/h）
SPEED_BUS = 20          # 公交速度上限（km/h）


def haversine(lon1, lat1, lon2, lat2, unit="km"):
    """球面距离（向量化）。"""
    R = 6371.0
    lon1, lat1, lon2, lat2 = map(np.radians, [lon1, lat1, lon2, lat2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
    c = 2 * np.arcsin(np.sqrt(a))
    d = R * c
    return d * 1000 if unit == "m" else d
```

```
def judge_mode_by_speed(avg_speed):
    """根据速度规则初步判定出行方式。"""
    if avg_speed < SPEED_OTHER:
        return "其他"
    elif avg_speed <= SPEED_BUS:
        return "公交"
    else:
        return "地铁或小汽车"


def mode_analysis():
    """执行出行方式识别完整流程。"""
    print("=" * 60)
    print("出行方式识别")
    print("=" * 60)

    # 步骤1：读取阶段3结果
    print("\n步骤1：读取阶段3结果")
    label_path = os.path.join(RESULT_DIR, "stage3_label.csv")
    if not os.path.exists(label_path):
        print("错误：请先运行 tr_stage3.py")
        return None
    df = pd.read_csv(label_path, dtype={"imsi": str, "time": str,
                                        "lac_id": str, "cell_id": str})
    df = df.sort_values(["imsi", "time"]).reset_index(drop=True)
    print(f"  数据：{len(df)} 行，{df['imsi'].nunique()} 用户")

    # 步骤2：出行段识别
    print("\n步骤2：出行段识别")
    df["is_travel"] = df["label"] == 1
    df["user_change"] = df["imsi"] != df["imsi"].shift(1)
    df["travel_change"] = df["is_travel"] != df["is_travel"].shift(1)
    df["seg_change"] = df["user_change"] | df["travel_change"]
    df["seg_id"] = df.groupby("imsi")["seg_change"].cumsum()
    travel_df = df[df["is_travel"]].copy()
```

```
    segments = travel_df.groupby(["imsi", "seg_id"]).agg(
        start_time=("time", "min"),
        end_time=("time", "max"),
        points=("time", "count"),
        start_lon=("lon", "first"),
        start_lat=("lat", "first"),
        end_lon=("lon", "last"),
        end_lat=("lat", "last"),
    ).reset_index()
    segments["start_dt"] = pd.to_datetime(segments["start_time"],
format="%Y%m%d%H%M%S", errors="coerce")
    segments["end_dt"] = pd.to_datetime(segments["end_time"], format="%Y%m%d%H%M%S",
errors="coerce")
    segments["duration_hours"] = (segments["end_dt"] -
segments["start_dt"]).dt.total_seconds() / 3600

    # 过滤点数<2
    segments = segments[segments["points"] >= 2].copy()
    print(f"  出行段数: {len(segments)}")

    # 步骤3: 计算平均速度
    print("\n步骤3: 计算出行段平均速度")
    avg_speeds = []
    for idx, seg in segments.iterrows():
        seg_points = travel_df[(travel_df["imsi"] == seg["imsi"]) &
                               (travel_df["seg_id"] == seg["seg_id"])].copy()
        seg_points = seg_points.sort_values("time").reset_index(drop=True)
        if len(seg_points) < 2:
            avg_speeds.append(np.nan)
            continue
        seg_points["time_dt"] = pd.to_datetime(seg_points["time"],
format="%Y%m%d%H%M%S", errors="coerce")
        seg_points["time_diff"] = seg_points["time_dt"].diff().dt.total_seconds()
        seg_points["prev_lon"] = seg_points["lon"].shift(1)
        seg_points["prev_lat"] = seg_points["lat"].shift(1)
        seg_points["dist_diff"] = haversine(seg_points["prev_lon"],
seg_points["prev_lat"],
                                            seg_points["lon"], seg_points["lat"])
```

```
        seg_points["speed"] = seg_points["dist_diff"] * 3600 / seg_points["time_diff"]
        valid = seg_points["speed"][(seg_points["speed"] > 0) & (seg_points["speed"] <
200)]
        avg_speeds.append(valid.mean() if len(valid) > 0 else np.nan)

    segments["avg_speed"] = avg_speeds
    segments = segments[segments["avg_speed"].notna()].copy()
    print(f"  有效出行段: {len(segments)}")
    print(f"  平均速度: {segments['avg_speed'].mean():.2f} km/h")

    # 步骤4：速度规则判定
    print("\n步骤4：速度规则判定")
    segments["mode_by_speed"] = segments["avg_speed"].apply(judge_mode_by_speed)
    print(f"  速度规则判定:")
    print(segments["mode_by_speed"].value_counts().to_string())

    # 步骤5：读取静态数据
    print("\n步骤5：读取出行方式静态数据（GBK）")
    mode_path = os.path.join(DATA_DIR, "出行方式静态数据.csv")
    mode_df = pd.read_csv(mode_path, encoding="gbk")
    if len(mode_df.columns) == 5:
        mode_df.columns = ["mode_type", "station_name", "line_name", "lon", "lat"]
    mode_df["lon"] = pd.to_numeric(mode_df["lon"], errors="coerce")
    mode_df["lat"] = pd.to_numeric(mode_df["lat"], errors="coerce")
    mode_df = mode_df.dropna(subset=["lon", "lat"]).drop_duplicates()
    subway_df = mode_df[mode_df["mode_type"].str.contains("地铁", na=False)].copy()
    bus_df = mode_df[mode_df["mode_type"].str.contains("公交", na=False)].copy()
    print(f"  地铁站点: {len(subway_df)}, 公交站点: {len(bus_df)}")

    subway_lons = subway_df["lon"].values
    subway_lats = subway_df["lat"].values
    bus_lons = bus_df["lon"].values
    bus_lats = bus_df["lat"].values

    # 步骤6：空间匹配
    print("\n步骤6：空间匹配")
    segments["subway_match_points"] = 0
    segments["bus_match_points"] = 0
```

```
    for idx, seg in segments.iterrows():
        seg_points = travel_df[(travel_df["imsi"] == seg["imsi"]) &
                               (travel_df["seg_id"] == seg["seg_id"])].copy()
        seg_points = seg_points.sort_values("time").reset_index(drop=True)
        subway_count = 0
        bus_count = 0
        for _, point in seg_points.iterrows():
            if (haversine(point["lon"], point["lat"], subway_lons, subway_lats,
unit="m") < MATCH_THRESHOLD).any():
                subway_count += 1
            if (haversine(point["lon"], point["lat"], bus_lons, bus_lats, unit="m") <
MATCH_THRESHOLD).any():
                bus_count += 1
        segments.at[idx, "subway_match_points"] = subway_count
        segments.at[idx, "bus_match_points"] = bus_count

    # 步骤7：最终判定
    print("\n步骤7：最终出行方式判定")
    def final_judge(row):
        if row["subway_match_points"] >= SUBWAY_MIN_POINTS:
            return "地铁"
        elif row["bus_match_points"] >= BUS_MIN_POINTS:
            return "公交"
        elif row["avg_speed"] > SPEED_BUS:
            return "小汽车"
        else:
            return "其他"
    segments["final_mode"] = segments.apply(final_judge, axis=1)
    print(f"  最终出行方式:")
    print(segments["final_mode"].value_counts().to_string())

    # 输出
    output_path = os.path.join(RESULT_DIR, "mode_result.csv")
    output_cols = ["imsi", "start_time", "end_time", "duration_hours", "points",
                   "start_lon", "start_lat", "end_lon", "end_lat",
                   "avg_speed", "mode_by_speed",
                   "subway_match_points", "bus_match_points", "final_mode"]
```

```
    segments[output_cols].to_csv(output_path, index=False, encoding="utf-8")
    print(f"\n输出：{output_path}")
    print(f"  行数：{len(segments)}")

    print("\n" + "=" * 60)
    print("出行方式识别完成！")
    print("=" * 60)
    return segments


if __name__ == "__main__":
    mode_analysis()
```

6.5 运行结果与解读

运行后展示每步统计，输出 mode_result.csv。

6.6 当堂练习

1. 运行 mode.py，记录最终出行方式分布；
2. 检查 mode_result.csv 的前20行，理解各字段含义。

6.7 课时小结

- mode.py 整合出行方式所有逻辑；
- 7步流程：读取数据→出行段识别→计算速度→速度规则→读取静态数据→空间匹配→最终判定；
- 输出 mode_result.csv。

课时7：出行方式结果分析与可视化

7.1 课时目标

分析出行方式结果，可视化方式分布、速度分布、时间分布。

7.2 完整代码（结果分析与可视化）

```
# ============================================================
# 课时7：出行方式结果分析与可视化
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

plt.rcParams["font.sans-serif"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"
FIG_DIR = os.path.join(RESULT_DIR, "figures")
os.makedirs(FIG_DIR, exist_ok=True)

df = pd.read_csv(os.path.join(RESULT_DIR, "mode_result.csv"),
                 dtype={"imsi": str, "start_time": str, "end_time": str})
print(f"出行段：{len(df)} 段")

# 图1：出行方式分布（饼图+柱状图）
print("生成图1：出行方式分布")
mode_counts = df["final_mode"].value_counts()
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
colors = ["red", "blue", "orange", "gray"]
axes[0].pie(mode_counts.values, labels=mode_counts.index, autopct="%1.1f%%",
            colors=colors, startangle=90)
axes[0].set_title("出行方式分布（饼图）")
axes[1].bar(mode_counts.index, mode_counts.values, color=colors, edgecolor="black")
axes[1].set_ylabel("出行段数")
axes[1].set_title("出行方式分布（柱状图）")
for i, v in enumerate(mode_counts.values):
    axes[1].text(i, v + 1, str(v), ha="center")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_mode_dist.png"), dpi=150, bbox_inches="tight")
```

```
plt.close()

# 图2：各方式速度分布（箱线图）
print("生成图2：各方式速度分布")
fig, ax = plt.subplots(figsize=(10, 6))
modes = ["地铁", "公交", "小汽车", "其他"]
data = [df[df["final_mode"] == m]["avg_speed"].values for m in modes]
bp = ax.boxplot(data, labels=modes, patch_artist=True)
for patch, color in zip(bp["boxes"], colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)
ax.set_ylabel("平均速度（km/h）")
ax.set_title("各出行方式平均速度分布")
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_mode_speed.png"), dpi=150, bbox_inches="tight")
plt.close()

# 图3：各方式开始时间分布
print("生成图3：各方式开始时间分布")
df["start_hour"] = pd.to_datetime(df["start_time"], format="%Y%m%d%H%M%S",
errors="coerce").dt.hour
fig, ax = plt.subplots(figsize=(12, 5))
for mode, color in zip(modes, colors):
    sub = df[df["final_mode"] == mode]
    hour_dist = sub["start_hour"].value_counts().reindex(range(24), fill_value=0)
    ax.plot(hour_dist.index, hour_dist.values, label=mode, color=color, linewidth=2)
ax.set_xlabel("开始小时")
ax.set_ylabel("出行段数")
ax.set_title("各出行方式开始时间分布")
ax.set_xticks(range(24))
ax.legend()
ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_mode_start_hour.png"), dpi=150,
bbox_inches="tight")
plt.close()

# 图4：速度规则 vs 最终方式（堆叠柱状图）
```

```
print("生成图4：速度规则 vs 最终方式")
cross = pd.crosstab(df["mode_by_speed"], df["final_mode"])
fig, ax = plt.subplots(figsize=(10, 6))
cross.plot(kind="bar", stacked=True, ax=ax, color=colors, edgecolor="black")
ax.set_xlabel("速度规则判定")
ax.set_ylabel("出行段数")
ax.set_title("速度规则 vs 最终出行方式")
ax.legend(title="最终方式")
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig(os.path.join(FIG_DIR, "fig_mode_cross.png"), dpi=150, bbox_inches="tight")
plt.close()

print(f"\n图片保存在：{FIG_DIR}")
print(f"  fig_mode_dist.png")
print(f"  fig_mode_speed.png")
print(f"  fig_mode_start_hour.png")
print(f"  fig_mode_cross.png")

# 统计摘要
print(f"\n{'='*60}")
print("出行方式统计摘要")
print(f"{'='*60}")
print(f"总出行段：{len(df)}")
for mode in modes:
    sub = df[df["final_mode"] == mode]
    if len(sub) > 0:
        print(f"  {mode}：{len(sub)} ({len(sub)/len(df)*100:.1f}%)，"
              f"平均速度{sub['avg_speed'].mean():.2f}km/h，"
              f"平均持续{sub['duration_hours'].mean():.2f}h，"
              f"涉及用户{sub['imsi'].nunique()}")
```

7.3 运行结果与解读

生成4张图：方式分布、速度分布、开始时间、速度规则vs最终方式。

7.4 课时小结

- 出行方式分布：其他最多，小汽车次之，公交/地铁较少；

- 速度分布：地铁最快，公交中等，小汽车较快，其他最慢；
- 开始时间：早高峰（7-9点）和晚高峰（17-19点）出行段多；
- 速度规则vs最终方式：空间匹配修正了部分误判。

课时8：参数调优与边界情况

8.1 课时目标

理解出行方式识别各参数的影响，处理边界情况。

8.2 知识点深度讲解

8.2.1 参数敏感性

1. **速度阈值**（12/20km/h）：影响其他/公交/地铁或小汽车的分界；
2. **匹配阈值**（500m）：影响匹配点数；
3. **地铁最小点数**（2）：影响地铁段数量；
4. **公交最小点数**（3）：影响公交段数量。

8.2.2 边界情况

1. **出行段只有2个点**：可能匹配不到足够站点；
2. **速度接近阈值**（12或20km/h）：可能误判；
3. **同时满足地铁和公交条件**：按优先级判为地铁；
4. **高速但不匹配站点**：判为小汽车；
5. **低速但匹配公交站**：仍判为其他（速度优先）。

8.3 完整代码（参数调优与边界情况）

```
# ============================================================
# 课时8：参数调优与边界情况
# ============================================================

import pandas as pd
import numpy as np
import os

RESULT_DIR = r"E:\东软教育\合作院校\河南师范大学\2026年\马静波大数据项目生成\交通时空大数据分析系统-14天授课笔记_第2版\result"

df = pd.read_csv(os.path.join(RESULT_DIR, "mode_result.csv"),
                 dtype={"imsi": str, "start_time": str, "end_time": str})

# 1. 速度阈值敏感性
print("=" * 60)
print("1. 速度阈值敏感性")
print("=" * 60)
for other_th in [10, 12, 15]:
    for bus_th in [18, 20, 25]:
        if other_th >= bus_th:
            continue
        def judge(s):
            if s < other_th: return "其他"
            elif s <= bus_th: return "公交"
            else: return "地铁或小汽车"
        tmp = df["avg_speed"].apply(judge)
        print(f"  其他<{other_th}, 公交<={bus_th}: "
              f"其他={(tmp=='其他').sum()}, 公交={(tmp=='公交').sum()}, "
              f"地铁或小汽车={(tmp=='地铁或小汽车').sum()}")

# 2. 匹配阈值敏感性
print(f"\n{'='*60}")
print("2. 匹配阈值敏感性（基于匹配点数近似分析）")
print(f"{'='*60}")
print(f"  地铁匹配点数分布:")
```

```
print(df["subway_match_points"].value_counts().sort_index().to_string())
print(f"  公交匹配点数分布:")
print(df["bus_match_points"].value_counts().sort_index().to_string())

# 3. 地铁最小点数敏感性
print(f"\n{'='*60}")
print("3. 地铁最小点数敏感性")
print(f"{'='*60}")
for min_p in [1, 2, 3, 4]:
    subway_count = (df["subway_match_points"] >= min_p).sum()
    print(f"  地铁≥{min_p}点: {subway_count} 段")

# 4. 公交最小点数敏感性
print(f"\n{'='*60}")
print("4. 公交最小点数敏感性")
print(f"{'='*60}")
for min_p in [1, 2, 3, 4, 5]:
    bus_count = (df["bus_match_points"] >= min_p).sum()
    print(f"  公交≥{min_p}点: {bus_count} 段")

# 5. 边界情况
print(f"\n{'='*60}")
print("5. 边界情况分析")
print(f"{'='*60}")

# 2点段
two_points = df[df["points"] == 2]
print(f"  2点段: {len(two_points)}")
print(f"    方式分布: {two_points['final_mode'].value_counts().to_dict()}")

# 速度接近12
near_12 = df[(df["avg_speed"] >= 10) & (df["avg_speed"] <= 14)]
print(f"  速度接近12(10-14): {len(near_12)}")
print(f"    方式分布: {near_12['final_mode'].value_counts().to_dict()}")

# 速度接近20
near_20 = df[(df["avg_speed"] >= 18) & (df["avg_speed"] <= 22)]
print(f"  速度接近20(18-22): {len(near_20)}")
```

```
print(f"    方式分布: {near_20['final_mode'].value_counts().to_dict()}")

# 同时满足地铁和公交
both = df[(df["subway_match_points"] >= 2) & (df["bus_match_points"] >= 3)]
print(f"  同时满足地铁和公交: {len(both)}")
print(f"    最终方式: {both['final_mode'].value_counts().to_dict()} (应为地铁)")

# 高速但不匹配站点
high_no_match = df[(df["avg_speed"] > 20) & (df["subway_match_points"] < 2)]
print(f"  高速(>20)但地铁匹配<2点: {len(high_no_match)}")
print(f"    最终方式: {high_no_match['final_mode'].value_counts().to_dict()}")

print(f"\n参数推荐:")
print(f"  其他速度上限: 12 km/h")
print(f"  公交速度上限: 20 km/h")
print(f"  匹配阈值: 500 m")
print(f"  地铁最小点数: 2")
print(f"  公交最小点数: 3")
```

8.4 运行结果与解读

展示参数敏感性分析和边界情况统计。

8.5 课时小结

- 速度阈值12/20是合理分界；
- 匹配阈值500m允许定位误差；
- 地铁≥2点、公交≥3点是合理阈值；
- 边界情况：2点段、速度接近阈值、同时满足地铁和公交；
- 参数调优可根据实际数据调整。

三、本日小结

1. **出行方式总览**：地铁、公交、小汽车、其他四类，速度规则+空间匹配；
2. **出行段识别**：连续出行点合并，cumsum实现段ID；
3. **速度规则**：<12其他，12-20公交，>20地铁或小汽车；
4. **静态数据**：GBK编码，地铁/公交站点位置；

5. **空间匹配**：出行段内与站点<500m的点数，地铁≥2点、公交≥3点；
6. **mode.py**：完整实现出行方式识别，输出 mode_result.csv；
7. **结果分析**：其他最多，地铁最快，早/晚高峰出行段多；
8. **参数调优**：12/20速度阈值、500m匹配阈值、地铁≥2/公交≥3是合理配置。

四、本日作业

1. 运行 mode.py，记录最终出行方式分布；
2. 运行可视化代码，查看4张图，写一段分析；
3. 思考：为什么"其他"类通常最多？（提示：城市内步行出行段多）
4. 预习 Day09：思考"如何用滑动窗口统计实时商圈热力？"